21

© 한상훈 NestJS 백엔드 마스터

NestJS 백엔드 마스터

PART 2

순환 의존, ModuleRef, 지연 로딩, Discovery

제2부 NestJS 핵심 · 5장 의존성 주입

서로를 필요로 하는 서비스, 실행 중에야 정해지는 주입



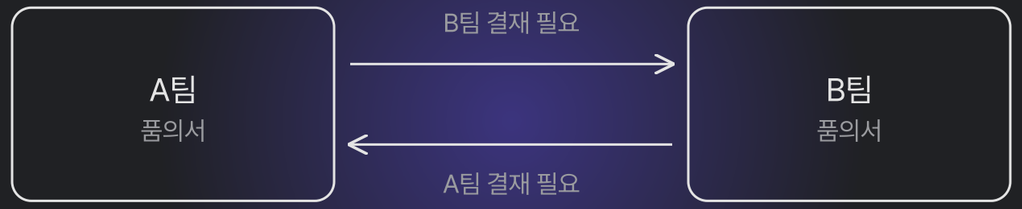
© 한상훈 NestJS 백엔드 마스터

서로의 결재를 기다리는 품의서

→ A팀 품의서: B팀 결재 필요

→ B팀 품의서: A팀 결재 필요

→ 누가 먼저 도장을 찍을지 미정 → 둘 다 정지



누가 먼저 도장을 찍을지 미정

둘 다 정지

© 한상훈 NestJS 백엔드 마스터

예매 확정 문자

→ 예매 흐름 마지막 단계: 확정 알림

→ 예매 서비스: 결제 끝나면 알림 서비스 호출

→ 알림 서비스: 문자에 넣을 구매자와 좌석을 예매 서비스에 조회

→ 예매 → 알림, 알림 → 예매

© 한상훈 NestJS 백엔드 마스터

예매를 받는 알림 서비스

notifications.service.ts · notifications.module.ts

```
@Injectable()
export class NotificationsService {
  constructor(private readonly reservations: ReservationsService) {}

  sendConfirmed(reservationId: number) {
    const r = this.reservations.findOne(reservationId);
    return `${r.buyer}님, ${r.seatId} 예매가 확정되었습니다`;
  }
}

@Module({
  imports: [ReservationsModule],
  providers: [NotificationsService],
  exports: [NotificationsService],
})
export class NotificationsModule {}
```

© 한상훈 NestJS 백엔드 마스터

알림을 받는 예매 서비스

reservations.service.ts

```
constructor(
  private readonly gateway: PaymentGateway,
  private readonly logger: TaggedLogger,
  private readonly notifications: NotificationsService,
) {
  this.logger.setTag('예매');
}

async reserve(seatId: string, amount: number, buyer: string) {
  const payment = await this.gateway.charge(amount);
  const id = this.nextId++;
  this.reservations.set(id, { id, seatId, buyer, approvalId: payment.approvalId });
  this.logger.log(`${seatId} 결제 완료`);
  return { id, seatId, ...payment, message: this.notifications.sendConfirmed(id) };
}
```

→ ReservationsModule: imports [NotificationsModule], exports [ReservationsService]

© 한상훈 NestJS 백엔드 마스터

뜨기도 전에 난 에러

터미널

```
$ node dist/apps/ticketing/main.js
file:///.../dist/apps/ticketing/main.js:864
  NotificationsService: () => (NotificationsService)
                               ^

ReferenceError: Cannot access 'NotificationsService' before initialization
```

→ Nest 로그 한 줄도 없음: Nest가 시작하기 전, 파일을 읽는 단계에서 중단

© 한상훈 NestJS 백엔드 마스터

순환 의존

→ 순환 의존(circular dependency): A가 B를 필요로 하고 B가 A를 필요로 하는 관계

→ 파일 순환: `reservations.service.ts` ↔ `notifications.service.ts`

→ 모듈 순환: `ReservationsModule` ↔ `NotificationsModule`

→ ES 모듈: 순환 import에서 한쪽은 아직 만들어지지 않은 클래스를 보게 됨

© 한상훈 NestJS 백엔드 마스터

양쪽에 forwardRef

```
// 모듈 양쪽
imports: [forwardRef(() => NotificationsModule)],
imports: [forwardRef(() => ReservationsModule)],

// 생성자 양쪽
@Inject(forwardRef(() => NotificationsService))
private readonly notifications: NotificationsService,

@Inject(forwardRef(() => ReservationsService))
private readonly reservations: ReservationsService,
```

→ `forwardRef(() => X)`: X를 지금 말고 나중에 꺼내라는 표시

→ 공식 문서의 방법: 모듈과 프로바이더 양쪽에

© 한상훈 NestJS 백엔드 마스터

그래도 같은 에러

터미널

```
file:///.../dist/apps/ticketing/main.js:864
  NotificationsService: () => (NotificationsService)
                               ^


ReferenceError: Cannot access 'NotificationsService' before initialization
```

© 한상훈 NestJS 백엔드 마스터

메타데이터가 클래스를 찾는 순간

→ 생성자 타입 NotificationsService → `design:paramtypes` 배열에 기록

→ 이 배열은 클래스를 정의하는 순간 계산

→ 그 순간 상대 파일은 아직 실행 전 → 클래스 없음 → `ReferenceError`

→ forwardRef가 늦춘 것: Nest의 주입 시점 / 못 늦춘 것: 메타데이터 계산 시점

© 한상훈 NestJS 백엔드 마스터

WrapperType으로 감싸기

```
// 메타데이터에 클래스 이름이 남지 않게 감싸는 타입 (ES 모듈의 순환 import 대응)
export type WrapperType<T> = T;

@Inject(forwardRef(() => NotificationsService))
private readonly notifications: WrapperType<NotificationsService>,

LOG [NestApplication] Nest application successfully started
{"id":1,"seatId":"A-3-15","approvalId":"fake-110000","amount":110000,"message":"김민수님, A-3-15 예매가 확정되었습니다"}
```

→ 공식 문서 SWC 절: "Wrapper type used to circumvent ESM modules circular dependency issue"

→ 공식 문서: "The order of instantiation is indeterminate." / "Avoid circular dependencies where possible"

데이터 블록: docs.nestjs.com/recipes/swc, docs.nestjs.com/fundamentals/circular-dependency (2026-10-26 기준), 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

생성자 대신 실행 중에 꺼내기

notifications.service.ts

```
@Injectable()
export class NotificationsService implements OnModuleInit {
  private reservations: ReservationsService;

  constructor(private readonly moduleRef: ModuleRef) {}

  async onModuleInit() {
    this.reservations = this.moduleRef.get(ReservationsService, { strict: false });
  }
```

→ `ModuleRef`: Nest 컨테이너에서 프로바이더를 직접 꺼내는 객체

→ `strict: false`: 이 모듈 밖의 프로바이더까지 검색

→ `onModuleInit`: 모듈의 프로바이더가 다 만들어진 뒤 Nest가 부르는 메서드

→ 알림 모듈 imports에서 예매 모듈 삭제

© 한상훈 NestJS 백엔드 마스터

get과 resolve

```
const a = await this.moduleRef.resolve(TaggedLogger, undefined, { strict: false });
const b = await this.moduleRef.resolve(TaggedLogger, undefined, { strict: false });
console.log(`resolve 두 번: 같은 객체인가? ${a === b}`);
this.moduleRef.get(TaggedLogger, { strict: false });
```

```
TaggedLogger #3 생성
TaggedLogger #4 생성
resolve 두 번: 같은 객체인가? false
InvalidClassScopeException [Error]: TaggedLogger is marked as a scoped provider. Request and
transient-scoped providers can't be used in combination with "get()" method.
Please, use "resolve()" instead.
```

© 한상훈 NestJS 백엔드 마스터

ModuleRef를 쓸 때

→ `get`: 싱글톤(DEFAULT) 프로바이더, 이미 만들어진 것

→ `resolve`: REQUEST·TRANSIENT, 부를 때마다 새로 생성

→ 맞는 곳: 무엇을 꺼낼지 실행 중에 정해질 때

→ 대가: 생성자에 안 보이는 의존, 빠진 프로바이더는 꺼내는 순간 발견

© 한상훈 NestJS 백엔드 마스터

알림이 예매를 모르게

```
// 알림 서비스: 예매 서비스 대신 값을 인자로
sendConfirmed(channel: string, to: string, seatId: string) { ... }

// 예매 서비스: 필요한 값을 넘김
const message = this.notifications.sendConfirmed(channel, buyer, seatId);
return { id, seatId, ...payment, message };
```

→ NotificationsModule imports: 예매 모듈 없음

→ forwardRef, WrapperType, ReservationsModule의 exports 삭제

© 한상훈 NestJS 백엔드 마스터

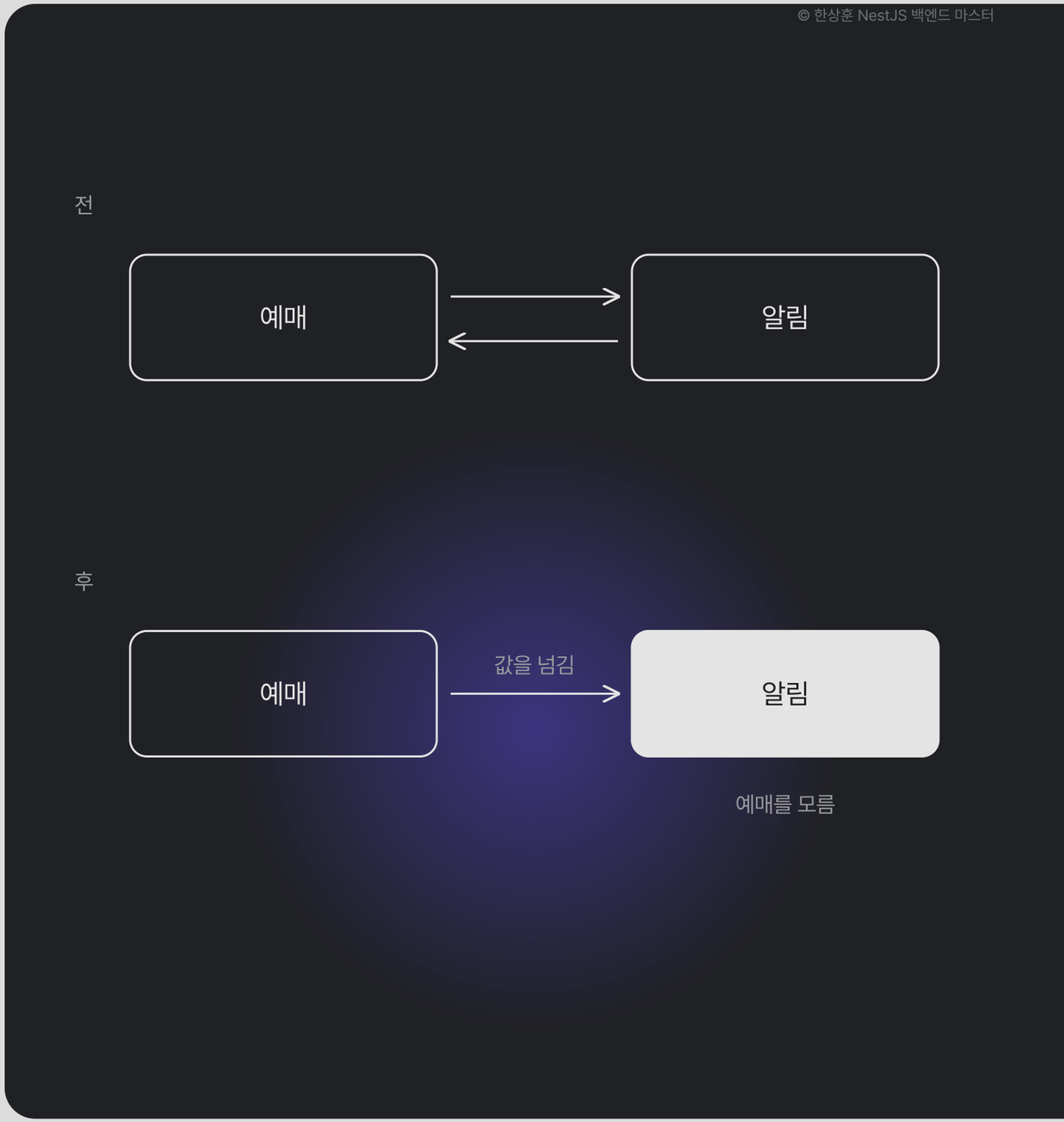
한 방향이 된 화살표

→ 전: 예매 ⇄ 알림

→ 후: 예매 → 알림 (알림은 예매를 모름)

→ 알림 서비스가 필요한 것: 받는 사람, 좌석, 채널, 예매 서비스가 이미 가진 값

→ 다른 방법: 공통 부분을 세 번째 서비스로 분리, 이벤트로 알림



© 한상훈 NestJS 백엔드 마스터

채널마다 다른 발송

→ 예매 확정 알림: 문자, 메일, 앱 푸시

→ 흔한 구현: 서비스 안의 switch 또는 채널 목록 배열

→ 채널 클래스를 추가하고 목록 수정을 빠뜨리면: "지원하지 않는 채널"

→ 목록을 사람이 관리하지 않고 부팅 때 찾아서

© 한상훈 NestJS 백엔드 마스터

채널 표시 데코레이터

sms.sender.ts

```ts
import { DiscoveryService } from '@nestjs/core';

export const NotificationChannel = DiscoveryService.createDecorator<string>();

@Injectable()
@NotificationChannel('sms')
export class SmsSender implements ChannelSender {
  send(to: string, text: string) {
    return `[문자 → ${to}] ${text}`;
  }
}
```

→ EmailSender('email'),
PushSender('push')도 같은 모양

© 한상훈 NestJS 백엔드 마스터

부팅 때 채널 모으기

notifications.service.ts

```
constructor(private readonly discovery: DiscoveryService) {}

onModuleInit() {
  const wrappers = this.discovery.getProviders({ metadataKey: NotificationChannel.KEY });
  for (const wrapper of wrappers) {
    const name = this.discovery.getMetadataByDecorator(NotificationChannel, wrapper)!;
    this.channels.set(name, wrapper.instance);
  }
  console.log(`알림 채널: ${[...this.channels.keys()].join(', ')}`);
}
```

→ Discovery 서비스: 등록된 프로바이더·컨트롤러를 찾아보는 Nest 내장 서비스

→ NotificationsModule imports: `DiscoveryModule`

© 한상훈 NestJS 백엔드 마스터

채널 네 번 요청

서버 로그와 응답

```
알림 채널: sms, email, push

{"id":1,...,"message":"[문자 → 김민수] A-3-15 예매가 확정되었습니다"}
{"id":2,...,"message":"[메일 → 김민수] A-3-15 예매가 확정되었습니다"}
{"id":3,...,"message":"[앱 푸시 → 김민수] A-3-15 예매가 확정되었습니다"}
{"id":4,...,"message":"지원하지 않는 채널: kakao"}
```

© 한상훈 NestJS 백엔드 마스터

채널 하나 추가

notifications.module.ts 변경

```
 import { EmailSender } from './email.sender.js';
+import { PushSender } from './push.sender.js';

 @Module({
   imports: [DiscoveryModule],
-  providers: [NotificationsService, SmsSender, EmailSender],
+  providers: [NotificationsService, SmsSender, EmailSender, PushSender],
```

→ 바뀐 것: 새 파일 `push.sender.ts`, providers 한 줄

→ NotificationsService: 변경 없음

© 한상훈 NestJS 백엔드 마스터

무거운 영수증 모듈

```
// receipts/receipts.service.ts
// 무거운 PDF 라이브러리를 불러오는 데 0.8초가 걸린다고 가정한 자리
const started = Date.now();
while (Date.now() - started < 800) {}
console.log('영수증 모듈 코드 불러옴 (0.8초)');

@Injectable()
export class ReceiptsService {
  render(reservationId: number) {
    return `영수증 PDF: 예매 ${reservationId}번`;
  }
}
```

→ 0.8초: 실제 라이브러리 대신 시간만 흉내 낸 가정

© 한상훈 NestJS 백엔드 마스터

LazyModuleLoader

reservations.controller.ts

```ts
@Get(':id/receipt')
async receipt(@Param('id') id: string) {
  const { ReceiptsModule } = await import('../receipts/receipts.module.js');
  const { ReceiptsService } = await import('../receipts/receipts.service.js');
  const moduleRef = await this.lazyModuleLoader.load(() => ReceiptsModule);
  return moduleRef.get(ReceiptsService).render(+id);
}
```

○ ○ ○ 터미널

```
$ ls dist/apps/ticketing
0.main.js  0.main.js.map  1.main.js  1.main.js.map  main.js  main.js.map
```

→ 동적 import: 빌드가 영수증 코드를 별도 파일로 분리

→ 지연 로딩 모듈(lazy-loaded module): 서버가 뜬 뒤 필요할 때 등록하는 모듈

© 한상훈 NestJS 백엔드 마스터

기동 시간과 첫 호출

방식	기동	영수증 첫 호출	두 번째
정적 import	1,858~1,955ms	3~4ms	1~4ms
지연 로딩	1,109~1,179ms	807~813ms	1~2ms

→ 기동: 프로세스 시작부터 첫 응답까지, 3회씩

데이터 블록: 지연 로딩 기동 1,165/1,109/1,179ms, 첫 호출 807/807/813ms · 정적 import 기동 1,924/1,955/1,858ms, 첫 호출 3/4/4ms, 2026-10-26 강사 PC 실측(Node v24.11.0), 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

지연 로딩의 제약

→ "Lazy loaded modules are cached on the first LazyModuleLoader#load method invocation."

→ 지연 로딩 모듈의 컨트롤러, 게이트웨이: 라우트로 등록 안 됨

→ "Lifecycle hook methods are not invoked in lazy loaded modules and services."

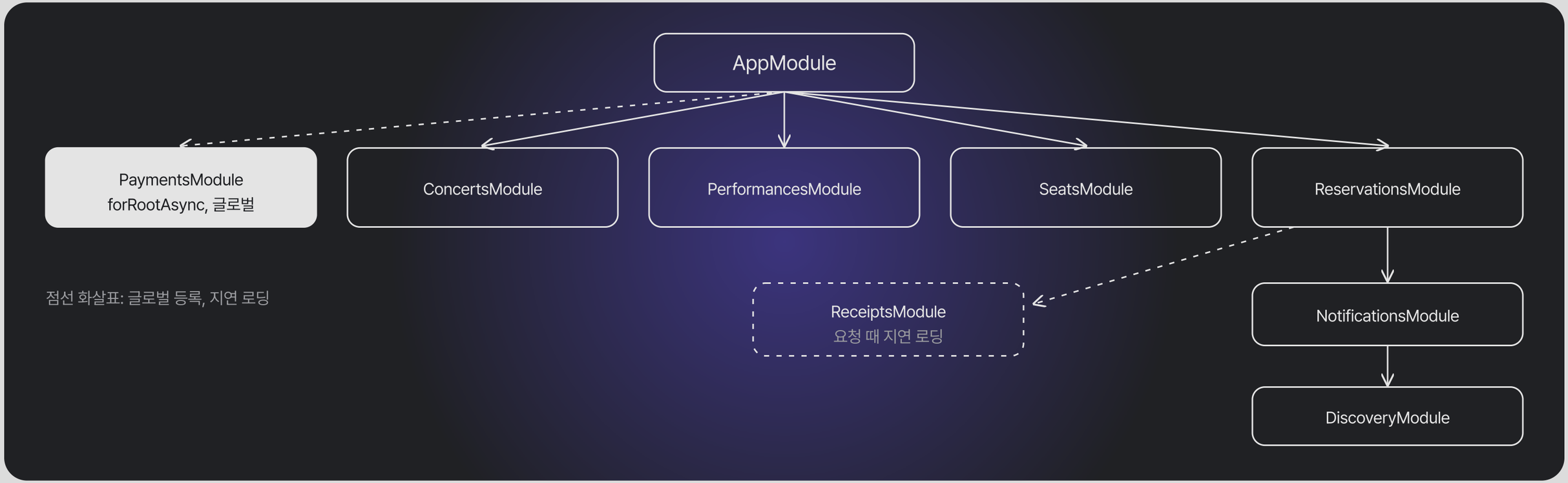
→ 맞는 곳: 서버리스처럼 기동 시간이 비용인 곳, 드물게 쓰는 무거운 기능

데이터 블록: docs.nestjs.com/fundamentals/lazy-loading-modules (2026-10-26 기준), 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

5장 끝의 모듈 그래프

→ AppModule → PaymentsModule(forRootAsync, 글로벌)
→ AppModule → ConcertsModule, PerformancesModule, SeatsModule, ReservationsModule
→ ReservationsModule → NotificationsModule → DiscoveryModule
→ ReservationsModule ⇢ ReceiptsModule (요청 때 지연 로딩)
→ 모든 화살표가 한 방향



© 한상훈 NestJS 백엔드 마스터

순환보다 값 넘기기

→ forwardRef + WrapperType: 띄울 수는 있음

→ ModuleRef: 생성자에서 의존이 사라짐

→ 값을 넘기는 구조: 순환 자체가 없음

"서로를 부르는 두 서비스는 forwardRef로 잇기 전에,
한쪽이 값을 넘기게 바꿀 수 없는지부터 봅니다."